SERVIÇO PÚBLICO FEDERAL · MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE VIÇOSA · UFV
CAMPUS FLORESTAL

Trabalho 1 - PAA

ALMIRO FURTADO DE MIRANDA NETO [6554]
SAMUEL NUNES FAUSTINO [6553]
JOÃO GABRIEL BARBOSA SCHERER [6551]
VITOR COSTA CARDOSO [6589]
JOAQUIM MADEIROS BASTOS CORDEIRO [6584]

Florestal - MG
2026

Sumário

1. Introdução **3**
2. Estrutura dos Arquivos **3**
3. Desenvolvimento **4**
3.1 Busca Por Um Caminho Válido 4
3.2 Busca Por Todos Os Caminhos Válidos 6
3.3 Busca Em Um Labirinto Sem Bordas 8
3.4 Impressão Criativa Dos Caminhos 8
3.4.1 Função printaMatriz 9
3.4.2 Função printCriativo 10
3.5 Gerador de Labirintos 11
3.5.1 Função criador 12
3.5.2 Função inicializador 12
3.5.3 Função printaMatrizGerador 13
3.5.4 Função escritor 13
3.5.5 Função gerador 14
3.6 Modo História Interativo 18
3.7 Main 19
4. Compilação e Execução **19**
5. Resultados de Execução **19**
5.1 Menu 21
5.2 Teste 1 22
5.3 Teste 2 24
5.4 Teste 3 24
5.5 Teste Gerado 25
6. Análise de Complexidade **26**
6.2 Análise da função escolheCaminho 26
6. 2 Análise da função encontraTodosCaminhos 27
6.3 Análise da função Cobrinha 28
6.4 Nível máximo atingido 29
7. Conclusão **30**
8. Referências **30**

1. Introdução

Este trabalho foi desenvolvido na disciplina de Projeto e Análise de Algoritmos (PAA). O objetivo da atividade é aplicar, em um problema concreto, a técnica de backtracking, que constrói a solução por etapas e, sempre que uma escolha leva a um beco sem saída, desfaz essa escolha e tenta uma alternativa.

O problema proposto é o de um aventureiro preso em um labirinto. O labirinto é representado por uma matriz, em que cada posição corresponde a uma sala: salas livres, paredes, chaves, a entrada do aventureiro e o baú do tesouro. A tarefa consiste em encontrar um caminho possível da posição inicial do aventureiro até o tesouro, passando pelas chaves existentes no mapa e sem repetir salas dentro de um mesmo caminho. Quando o aventureiro chega ao baú sem ter coletado todas as chaves, esse caminho é descartado e a busca continua por rotas alternativas.

O trabalho teve dois objetivos. O primeiro foi implementar, em conjunto, o código principal: a leitura do labirinto a partir de um arquivo de texto e o algoritmo de backtracking que encontra um caminho válido do aventureiro até o tesouro, coletando as chaves. O segundo foi o desenvolvimento, por cada integrante do grupo, de uma funcionalidade extra, que amplia o programa original.

2. Estrutura dos Arquivos

O projeto está dividido em pastas, uma para cada módulo, além dos arquivos de teste e de compilação.

- **Main/ (main.c):** programa principal, com o menu de ações que chama os demais módulos. O executável gerado pelo Makefile também fica nesta pasta.
- **Labirinto/ (labirinto.c, labirinto.h):** representação do labirinto (matriz de Sala e Posicao), algoritmos de backtracking e a impressão criativa no terminal.
- **ListaCaminhos/ (listaCaminhos.c, listaCaminhos.h):** lista encadeada usada para armazenar todos os caminhos válidos encontrados.
- **Cores/:** funções de cor do terminal (códigos ANSI), usadas na impressão do labirinto e das animações.
- **GeradorDeLabirintos/ (gerador.c, gerador.h):** geração de labirintos novos a partir de parâmetros e escrita no arquivo gerado.txt.

- **ModoHistoria/ (historia.c, historia.h):** modo história interativo, baseado no mito de Teseu e o Minotauro.
- **Testes/ (teste1.txt, teste2.txt, teste3.txt, gerado.txt):** labirintos de entrada. Os três primeiros são os casos de teste do trabalho e gerado.txt é a saída do gerador.
- **Makefile:** regras de compilação e execução do programa.

3. Desenvolvimento

3.1 Busca Por Um Caminho Válido

O problema de encontrar um caminho da sala inicial até o baú, coletando todas as chaves necessárias e sem repetir salas, foi resolvido com backtracking, na função recursiva escolheCaminho. A ideia é construir o caminho passo a passo: a cada sala, o aventureiro escolhe um vizinho livre e segue adiante. Se chegar a um beco sem saída, desfaz a última escolha e tenta outra direção. A busca termina na primeira solução encontrada ou quando todas as possibilidades se esgotam.

O labirinto é uma matriz de Sala, que guarda o conteúdo da célula e uma marca de visitada, e as coordenadas são representadas por Posicao (Figura 1). O caminho em construção fica em um vetor de posições, que funciona como uma pilha: a posição entra ao avançar e sai ao voltar. Quando a função retorna true, este vetor já contém o caminho completo, pronto para ser impresso.



```
typedef struct {
    char conteudo;
    bool visitado;
} Sala;

typedef struct{
    int x;
    int y;
} Posicao;
```

Figura 1: Struct Sala e Posicao.

O ponto central do algoritmo é o controle de visitação: a sala é marcada como visitada ao ser alcançada e só volta a ser liberada quando todos os seus

vizinhos já foram testados sem sucesso (Figura 2). Isso garante a restrição de não passar duas vezes pela mesma sala dentro de um caminho, mas permite que a mesma sala seja reutilizada em outro caminho, o que é necessário para que o backtracking explore todas as alternativas. O mesmo vale para as chaves: o contador é incrementado ao entrar numa sala com chave e decrementado ao voltar, de modo que a contagem sempre reflete apenas o caminho atual.



Figura 2: Término sem sucesso da exploração de uma sala em escolheCaminho.

Outro trecho que merece atenção é o tratamento do baú (Figura 3). Chegar até ele não basta: se o aventureiro ainda não tiver todas as chaves, esse caminho é descartado e a busca continua por outras rotas, que talvez passem antes pelas chaves que faltam.



Figura 3: Tratamento do baú na função escolheCaminho.

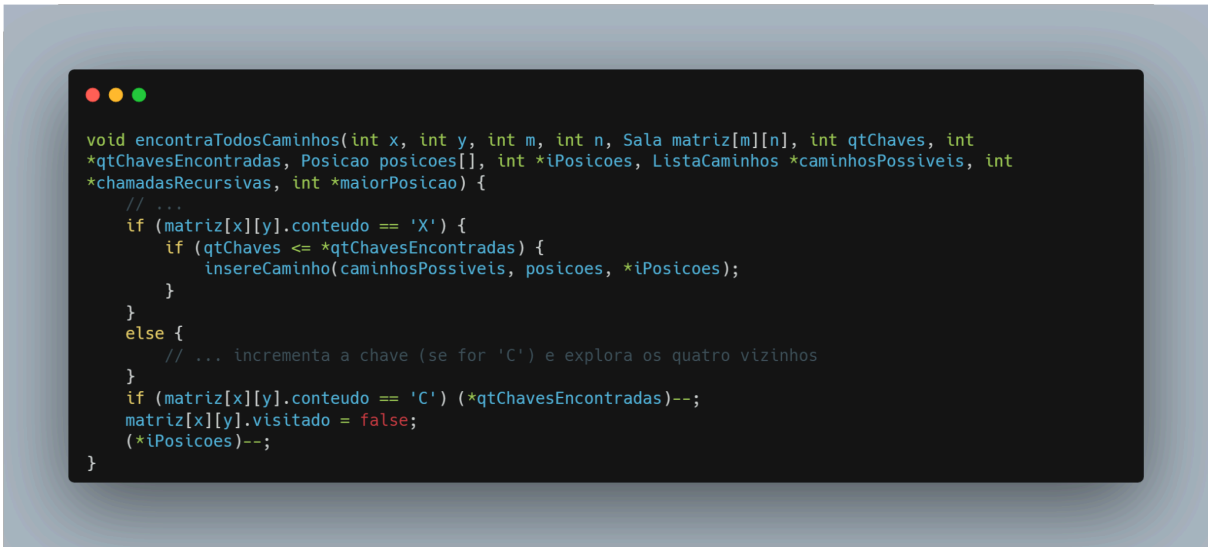
Decisões e limitações: a ordem de exploração dos vizinhos é fixa (esquerda, direita, baixo, cima), então o caminho retornado depende dessa ordem e não é necessariamente o menor. Os parâmetros chamadasRecursivas e maiorPosicao são passados por referência para registrar o total de chamadas e o nível máximo de

recursão sem alterar a lógica da busca. Por ser backtracking, o tempo no pior caso é exponencial no número de salas livres, o que aparece no Teste 3, que exigiu mais de 108 milhões de chamadas (detalhado na seção 5).

3.2 Busca Por Todos Os Caminhos Válidos

Como extra, foi implementada a função encontraTodosCaminhos, que registra todos os caminhos válidos em vez de parar no primeiro. Ela mantém a estrutura de escolheCaminho, com três adaptações:

- **Tipo de retorno.** A função passou a ser void, pois não há mais um resultado que precise interromper a busca.
- **Continuação após uma solução.** Ao chegar ao baú com as chaves suficientes, o caminho atual é salvo e a recursão continua normalmente, para que as demais combinações sejam exploradas (Figura 4). O baú funciona como ponto final: a busca não segue a partir dele.
- **Desfazer em toda chamada.** Como nenhuma chamada interrompe a busca, as escolhas (chave, marca de visitada, posição no vetor) são desfeitas ao final de toda chamada, e não apenas nos casos de fracasso.



```
void encontraTodosCaminhos(int x, int y, int m, int n, Sala matriz[m][n], int qtChaves, int
*qtChavesEncontradas, Posicao posicoes[], int *iPosicoes, ListaCaminhos *caminhosPossiveis, int
*chamadasRecursivas, int *maiorPosicao) {
    // ...
    if (matriz[x][y].conteudo == 'X') {
        if (qtChaves <= *qtChavesEncontradas) {
            insereCaminho(caminhosPossiveis, posicoes, *iPosicoes);
        }
    }
    else {
        // ... incrementa a chave (se for 'C') e explora os quatro vizinhos
    }
    if (matriz[x][y].conteudo == 'C') (*qtChavesEncontradas)--;
    matriz[x][y].visitado = false;
    (*iPosicoes)--;
}
```

Figura 4: Registro do caminho e reversão das alterações na função encontraTodosCaminhos.

Para guardar os caminhos encontrados foi criado o módulo ListaCaminhos. Como não se sabe de antemão quantas soluções existem nem qual o tamanho de

cada uma, um vetor fixo não seria adequado. Optou-se por uma lista encadeada simples com nó cabeça e ponteiro para o final (Figura 5). Cada nó guarda um vetor alocado dinamicamente com as posições de um caminho, a quantidade de posições desse vetor e o ponteiro para o próximo nó. A lista mantém os ponteiros para o início e para o final, além do total de caminhos armazenados. O ponteiro para o final permite inserir em tempo constante, sem percorrer a lista, e o nó cabeça simplifica a inserção, pois elimina o caso especial da lista vazia.



```c
typedef struct NoCaminho {
    Posicao *vetorCaminho;
    int tamanhoCaminho;
    struct NoCaminho *prox;
} NoCaminho;

typedef struct {
    NoCaminho *inicio, *final;
    int qtdCaminhos;
} ListaCaminhos;
```

Figura 5: Struct NoCaminho e ListaCaminhos.

Uma decisão importante está na inserção (Figura 6). Apesar de insereCaminho receber o vetor posicoes por parâmetro, ela não guarda esse ponteiro: aloca um novo vetor e copia as posições para ele. Isso é necessário porque posicoes é reaproveitado e sobrescrito durante todo o backtracking. Se os nós apenas apontassem para ele, todos os caminhos armazenados compartilhariam o mesmo vetor, no qual o conteúdo muda a cada passo da busca.



```c
void insereCaminho(ListaCaminhos *lista, Posicao caminho[], int tamanho) {
    lista->final->prox = (NoCaminho*) malloc(sizeof(NoCaminho));
    lista->final = lista->final->prox;
    lista->final->vetorCaminho = (Posicao*) malloc(tamanho * sizeof(Posicao));
    for( int i = 0; i < tamanho; i++) {
        lista->final->vetorCaminho[i] = caminho[i];
    }
    lista->final->tamanhoCaminho = tamanho;
    lista->final->prox = NULL;
    (lista->qtdCaminhos)++;
}
```

Figura 6: Função insereCaminho.

O módulo ainda implementa as operações básicas de uma lista: inicialização, verificação de lista vazia, impressão, consulta à quantidade de caminhos e liberação de memória. A liberação percorre a lista desalocando os vetores e os nós, para evitar vazamentos. Além da busca completa, a lista é usada pelo Modo História Interativo (seção 2.6), que a consulta para validar os movimentos do jogador.

Decisões e limitações: armazenar todos os caminhos tem custo de memória proporcional ao número de soluções multiplicado pelo tamanho de cada uma, o que pode crescer muito em labirintos abertos. Além disso, a busca completa precisa percorrer todo o espaço de possibilidades, sem o corte antecipado da busca simples, e por isso é mais custosa (comparação na seção 5).

3.3 Busca Em Um Labirinto Sem Bordas

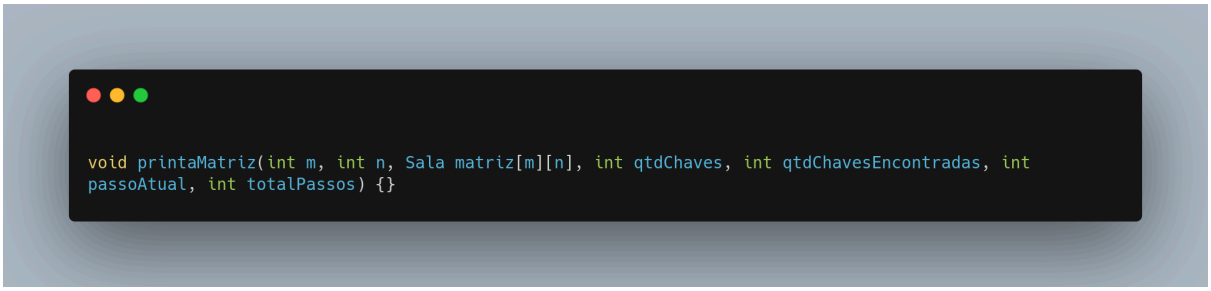
Em relação ao algoritmo de escolheCaminho, para que o algoritmo funcionasse sem bordas, isto é, dando a volta se fosse para fora das dependências do mapa, foi necessário retirar do algoritmo a restrição relacionada ao tamanho do mapa (a verificação se o personagem caminhou para além do limite da matriz), agora que o personagem pode ultrapassar os limites do mapa para garantir que ele irá para o outro lado a chamada recursiva deve ser módulo da medida do respectivo lado da matriz, assim quando ela passar essa conta colocará o personagem do outro lado.

3.4 Impressão Criativa Dos Caminhos

As funções de busca (escolheCaminho, encontraTodosCaminhos e Cobrinha) resolvem o labirinto apenas de forma lógica, guardando o caminho encontrado no vetor posicoes. Este capítulo descreve as duas funções que mostram esse resultado ao usuário no terminal. A primeira, printaMatriz, desenha o labirinto em um instante (um “quadro”), com cores e um painel de status. A segunda, printCriativo, usa a printaMatriz repetidamente para animar o percurso, passo a passo.

A ideia é separar a **lógica** (encontrar o caminho) da **apresentação** (mostrá-lo), de modo que as funções de busca não precisam saber nada sobre cores, telas ou tempo de espera.

3.4.1 Função printaMatriz



```
void printaMatriz(int m, int n, Sala matriz[m][n], int qtdChaves, int qtdChavesEncontradas, int
passoAtual, int totalPassos) {}
```

Figura 7 - Assinatura da função.

A função recebe as dimensões do labirinto, m linhas e n colunas, e a própria matriz de Sala, que representa o labirinto. Recebe ainda qtdChaves, o total de chaves existentes, e qtdChavesEncontradas, a quantidade de chaves coletadas até o passo atual. Por fim, passoAtual indica o número do passo que está sendo exibido e totalPassos o total de passos do caminho.

A função percorre a matriz linha a linha e, para cada linha, realiza três etapas:

1. **Borda esquerda:** imprime o caractere │.
2. **Células:** para cada coluna, escolhe como desenhar a célula de acordo com o seu conteúdo.
3. **Borda direita e painel:** imprime │ e, nas quatro primeiras linhas, o painel lateral de status.

As cores são aplicadas por meio das funções aplicarCorFundo, aplicarCorTexto e resetarCores, do módulo Cores, que encapsulam os códigos ANSI de cor e deixam o código de impressão mais legível. Ao final de cada célula, resetarCores() é chamada. Isso impede que a cor de uma célula “vaze” para a seguinte ou para o restante do texto.

Cada célula ocupa dois caracteres de largura. Isso compensa o fato de os caracteres do terminal serem mais altos do que largos, e as células ficam com formato aproximadamente quadrado. Cada tipo de conteúdo é desenhado de uma forma:

- **Caminho já percorrido (`#`):** dois espaços com fundo verde.
- **Chave (`C`):** a letra C em texto amarelo.

- **Parede (`1`):** dois espaços com fundo azul.
- **Saída (`X`) e entrada (`A`):** a letra correspondente, com fundo e texto magenta.
- **Corredor livre (demais caracteres, como `0`):** o próprio caractere, sem cor.
- **Painel de status:** Ao lado do labirinto, nas linhas de 0 a 3, a função exibe um pequeno painel com informações da execução, como no exemplo abaixo:

Para as paredes e o caminho percorrido, a função imprime apenas espaços com cor de fundo, formando blocos sólidos. Para chaves, entrada e saída, ela imprime o caractere, o que as destaca visualmente.



Figura 8 – Labirinto exibido no terminal.

3.4.2 Função printCriativo

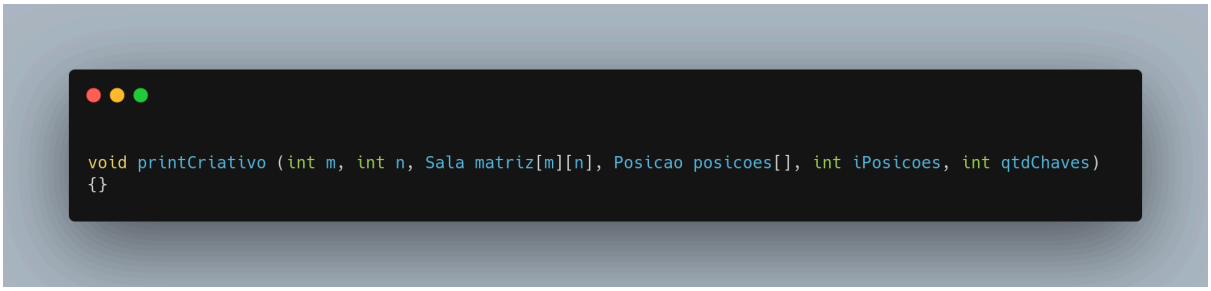


Figura 9 - Assinatura da função.

Os parâmetros m, n e matriz fornecem as dimensões e o conteúdo do labirinto. O vetor posicoes guarda as coordenadas do caminho encontrado, em ordem, e iPosicoes indica quantas posições estão armazenadas nele. Por fim, qtdChaves informa o total de chaves do labirinto.

A função recebe o caminho já calculado e o reproduz como uma animação:

1. **Inicialização:** passo começa em 1 e qtdChavesEncontradas em 0. O total de passos é iPosicoes - 2, pois a primeira posição (entrada) e a última (saída) não contam como passos intermediários.
2. **Laço de animação:** o for vai de i = 1 até i < iPosicoes - 1, ou seja, percorre apenas as posições intermediárias, preservando a entrada e a saída com suas cores originais. A cada iteração:
   - limparTela() apaga o quadro anterior, de modo que cada quadro é desenhado sobre o anterior, sem que o terminal “role” para baixo;
   - se a posição atual contém uma chave ('C'), o contador de chaves é incrementado;
   - a célula é marcada como '#', o que a faz aparecer em verde na impressão;
   - printaMatriz(...) desenha o labirinto com o estado atualizado e o painel de status;
   - passo é incrementado;
   - PAUSA(450) espera 450 ms antes do próximo quadro.
3. **Finalização:** ao terminar o laço, é impressa uma linha em branco para separar a animação do restante da saída do programa.

O resultado é um rastro verde que “cresce” a cada 450 ms, saindo da entrada, passando pelas chaves e chegando à saída. O contador de chaves e o número do passo no painel são atualizados em tempo real.

A printCriativo não desenha nada por conta própria: ela apenas prepara o estado (marca a célula, atualiza contadores) e delega o desenho à printaMatriz. Assim, a printCriativo fica responsável por controlar a sequência e o tempo da animação, enquanto a printaMatriz desenha um quadro do labirinto e o painel.

3.5 Gerador de Labirintos

O TAD Gerador de Labirintos contém as funções responsáveis por, a partir de parâmetros, criar um labirinto novo e escrevê-lo em um arquivo chamado gerado.txt. Assim, é composto pelos procedimentos criador, inicializador, printmatrizGerador, escritor e gerador, o qual é responsável por utilizar todos os outros anteriores para cumprir o objetivo. Todos eles serão explicados adiante.

3.5.1 Função criador

Essa função recebe dois parâmetros x e y, que correspondem a largura e altura do labirinto. Utilizando essas informações ela cria o espaço na memória correspondente a esse espaço, retornando uma matriz.



```
char** criador(int x, int y) {
    char **matriz = (char **)malloc(x * sizeof(char *));
    for (int i = 0; i < x; i++) {
        matriz[i] = (char *)malloc(y * sizeof(char));
    }
    return matriz;
}
```

Figura 10: Imagem da Função criador

3.5.2 Função inicializador

Como dados adquire x e y, que possuem o mesmo significado das variáveis anteriores e também a própria matriz, para colocar em cada posição o valor ‘0’. Serve para iniciar ou refazer o labirinto com diferentes dados.



```
void inicializador(char **matriz, int x, int y){
        for(int i = 0; i < x; i++){
        for(int j = 0; j < y; j++){
            matriz[i][j] = '0';
        }
    }
}
```

Figura 11: Imagem da Função inicializador

3.5.3 Função printaMatrizGerador

Possui igualmente os parâmetros anteriores, com a exceção de que o labirinto utilizado por ela já possui a inserção de chaves, paredes, baú e entrada, ou seja, no final. Após isso, exibe na tela a própria matriz de forma colorida, utilizando métodos pertencentes ao TAD Cores.



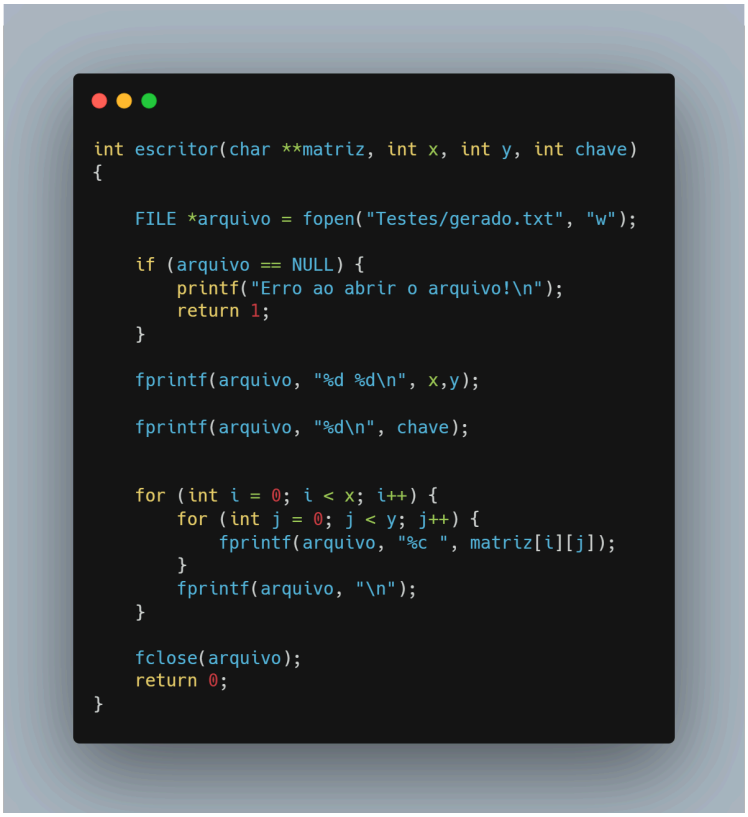
```
void printaMatrizGerador(int m, int n, char **matriz) {
    printf("\n");
    for (int i = 0; i < m; i++) {
        printf("| "); // Borda esquerda
        for (int j = 0; j < n; j++) {
            // Renderização customizada por elemento
            if (matriz[i][j] == 'C') {
                aplicarCorTexto(TEXTO_AMARELO);
                printf("C ");
            } else if (matriz[i][j] == '1') {
                aplicarCorFundo(FUNDO_AZUL);
                printf("  ");
            } else if (matriz[i][j] == 'X' || matriz[i][j] == 'A') {
                aplicarCorFundo(FUNDO_MAGENTA);
                aplicarCorTexto(TEXTO_MAGENTA);
                printf("%c ", matriz[i][j]);
            } else {
                printf("%c ", matriz[i][j]);
            }
            resetarCores();
        }

        printf("|"); // Borda direita do labirinto
        printf("\n");
    }
    printf("\n");
}
```

Figura 12: Imagem da função printmatrizGerador

3.5.4 Função escritor

Além de obter as mesmas informações que printmatrizGerador, ainda recebe a quantidade de chaves. Tudo isso é feito para que o gerado.txt receba esses dados, segundo o padrão mencionado na documentação sobre os arquivos de teste.



```c
int escritor(char **matriz, int x, int y, int chave)
{

    FILE *arquivo = fopen("Testes/gerado.txt", "w");

    if (arquivo == NULL) {
        printf("Erro ao abrir o arquivo!\n");
        return 1;
    }

    fprintf(arquivo, "%d %d\n", x,y);

    fprintf(arquivo, "%d\n", chave);


    for (int i = 0; i < x; i++) {
        for (int j = 0; j < y; j++) {
            fprintf(arquivo, "%c ", matriz[i][j]);
        }
        fprintf(arquivo, "\n");
    }

    fclose(arquivo);
    return 0;
}
```

Figura 13: Imagem da função escritor

3.5.5 Função gerador

Como mencionado no começo do tópico, essa é a função principal do TAD Gerador de Labirinto, que indica como e em qual ordem as outras operações agirão.

Em primeiro lugar, é de grande importância ressaltar que um dos parâmetros escolhidos é chamado de peca, que é utilizado na confecção das paredes. A lógica por trás de sua utilização deve-se pela ideia de criar 4 tipos de formato que são compostos pelo caractere ‘1’, ou seja blocos que representam pequenos obstáculos. Dessa forma, com a sua inserção na matriz, estruturas maiores são criadas quando eles se unem, permitindo grande variedade, e peca representam justamente a quantidade desses blocos. Isso foi inspirado no Trabalho Prático 0, o qual para formar obras de artes grandes usa combinações de diferentes formatos compostos por asteriscos, para formar imagens.

Os formatos escolhidos dos pequenos pedaços das paredes foram:

→1. 1

→2. 11

→3. 1
1

→4. 1
1

Além dessa variável, a função recebe também x e y, que novamente significam as dimensões do labirinto e chave que corresponde a quantidade de chaves que serão inseridas nele. Assim, o algoritmo segue as seguintes etapas:
→função criador, reserva o espaço da matriz desejada;

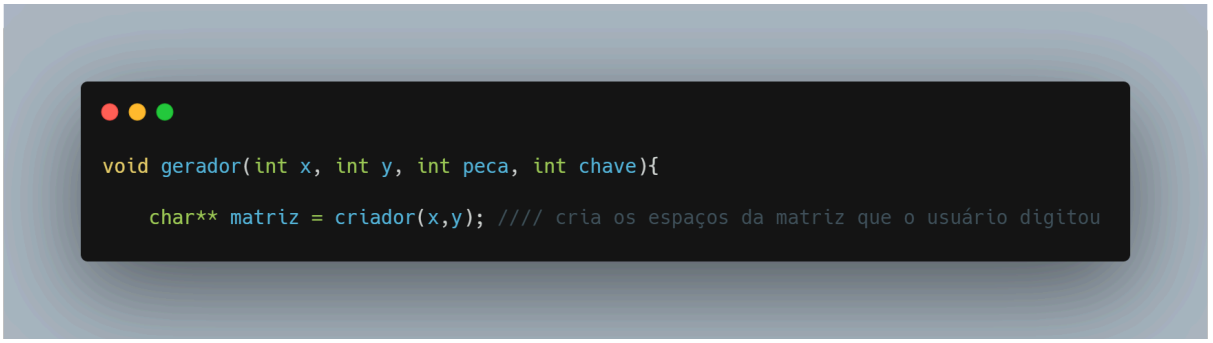
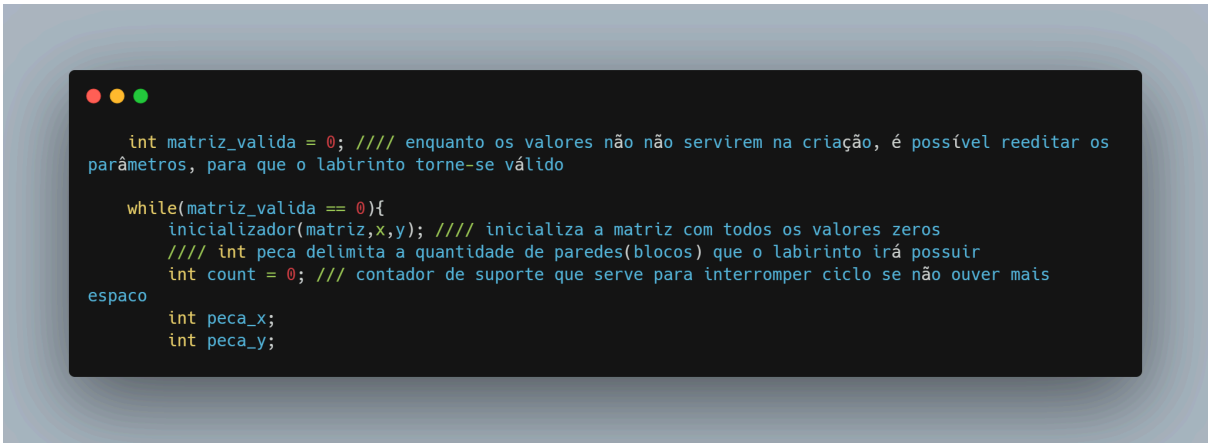


Figura 14: Papel de criador na função gerador

→uma variável de nome matriz_valida verifica se com os parâmetros são válidos para criar o labirinto, no momento da aplicação de cada característica nele. Caso uma delas não consiga ser efetuada por falta de espaço, o usuário pode editar posteriormente a redução de paredes e chaves, para tornar a matriz válida no próximo laço de repetição com os dados atualizados.Para isso, a matriz tem todas as suas posições preenchidas com ‘0’ para adquirir as novas informações. Isso revela que inicializador prepara a função no começo, mas também quando for ser refeita;
→com a matriz criada, o laço de repetição referente a matriz_válida inicializada, e o valor inicial das posições da matriz inseridos começa então a inserção das características do labirinto;



```
    int matriz_valida = 0; //// enquanto os valores não não servirem na criação, é possível reeditar os
parâmetros, para que o labirinto torne-se válido

    while(matriz_valida == 0){
        inicializador(matriz,x,y); //// inicializa a matriz com todos os valores zeros
        //// int peca delimita a quantidade de paredes(blocos) que o labirinto irá possuir
        int count = 0; /// contador de suporte que serve para interromper ciclo se não ouver mais
espaco
        int peca_x;
        int peca_y;
```

Figura 15: Papel de inicializador e matriz_valida na função gerador

→o algoritmo começa a inserção das peças. O que ocorre é a seleção aleatória do tipo de bloco que será inserido, bem como onde será inserido, de tal forma que não irá entrar em conflito com o espaço ocupado pelos outros. Existe uma margem de 50 vezes o qual o programa realiza este procedimento para cada tentativa. Se o programa não conseguir registrar uma parede, após todas as iniciativas, a variável matriz_válida avisa ao usuário que não foi possível a ação, para que depois o usuário manualmente altere a quantidade, para tornar o labirinto válido;

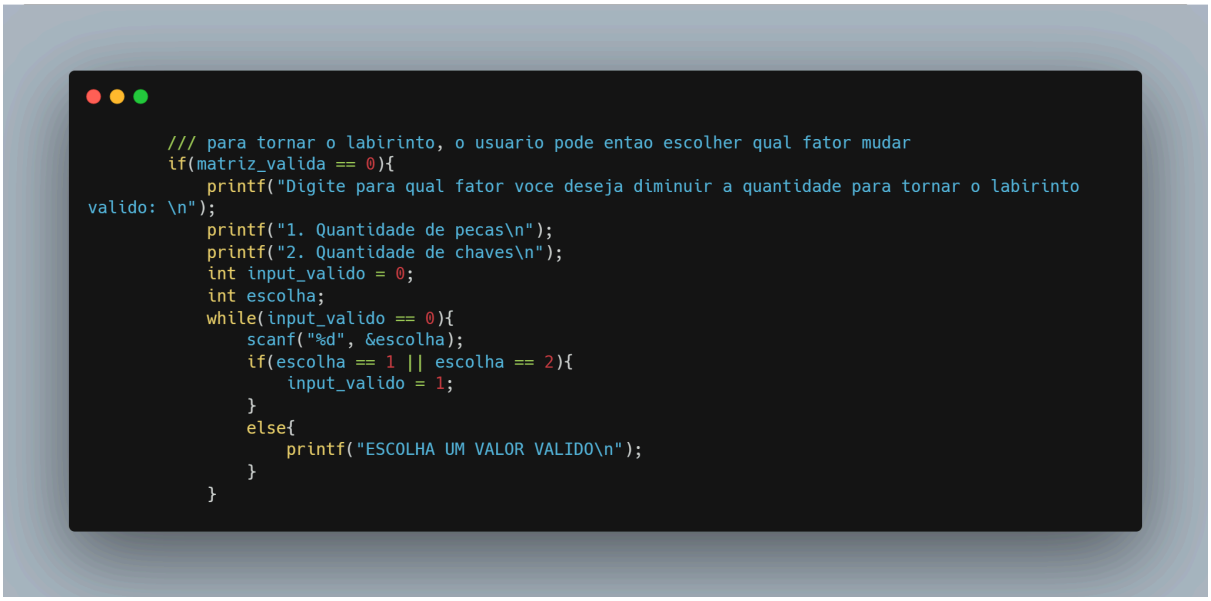


```
                else if(formato == 4){
                    if(count > 50){
                        printf("Nao foi possivel posicionar mais paredes.\n");
                        printf("Disparidade-> pecas desejadas(%d); pecas colocadas(%d)\n", peca,
contador_peca);
                        matriz_valida = 0;
                        break;
                    }
                        peca_x = rand() % (x - 1);
                        peca_y = rand() % (y - 1);
                        if (matriz[peca_x][peca_y] == '0' && matriz[peca_x + 1][peca_y + 1] == '0')
{
                            matriz[peca_x][peca_y] = '1';
                            matriz[peca_x + 1][peca_y + 1] = '1';
                            flag_p = 1;
                            contador_peca++;
                        }
                        else{
                            count++;
                        }
                    }
            }

            if(matriz_valida == 0){
                break;
            }
```

Figura 16: Parte da inserção de paredes

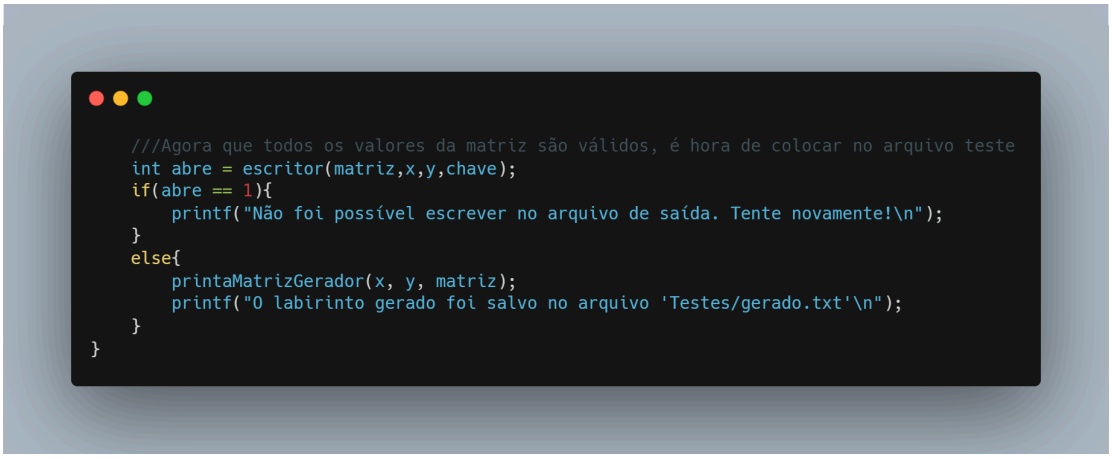
→logo após isso, o programa realiza o registro das outras informações seguindo a mesma lógica do procedimento de inserção das peças;
→ao final da função se a matriz for inválida, o usuário ainda pode reduzir a quantidade de chaves e peças, para que no próximo laço o labirinto consiga ser criado. Lembrando que o algoritmo ainda é atualizado sobre o novo valor dos dados anteriores, para ao caso de se precisar alterar ainda mais algum fator;



```
        /// para tornar o labirinto, o usuario pode entao escolher qual fator mudar
        if(matriz_valida == 0){
            printf("Digite para qual fator voce deseja diminuir a quantidade para tornar o labirinto
valido: \n");
            printf("1. Quantidade de pecas\n");
            printf("2. Quantidade de chaves\n");
            int input_valido = 0;
            int escolha;
            while(input_valido == 0){
                scanf("%d", &escolha);
                if(escolha == 1 || escolha == 2){
                    input_valido = 1;
                }
                else{
                    printf("ESCOLHA UM VALOR VALIDO\n");
                }
            }
```

Figura 17: Parte da modificação das variáveis do labirinto

→com o labirinto estando válido, é utilizado a função escritor para salvar a matriz no arquivo gerado.txt, e printmatrizGerador para exibi-la no terminal;



```
    ///Agora que todos os valores da matriz são válidos, é hora de colocar no arquivo teste
    int abre = escritor(matriz,x,y,chave);
    if(abre == 1){
        printf("Não foi possível escrever no arquivo de saída. Tente novamente!\n");
    }
    else{
        printaMatrizGerador(x, y, matriz);
        printf("O labirinto gerado foi salvo no arquivo 'Testes/gerado.txt'\n");
    }
}
```

Figura 18: Escrevendo e imprimindo labirinto

Observação: a forma pela qual o labirinto foi feito, não garante que conseguirá ser resolvido pelo aventureiro pelo fator da aleatoriedade. Essa escolha, porém, permite maior variedade, já que toda vez que o procedimento for colocado em funcionamento, sempre ocasionará em resultados diferentes, mesmo com os mesmos parâmetros. Além disso, a quantidade da variável numérica peca, quando a matriz estiver cheia, pode precisar de uma quantidade diferente de redução a cada vez que gerador opera, pois os blocos podem ter preenchido um determinado espaço de diferentes formas de acordo com o formato e o local os quais foram sendo inseridos. Ou seja, não só o total de paredes desejadas podem tornar uma matriz inválida, mas também os tipos e os locais de cada bloco colocado, o que gera necessidades de correção diferentes para cada laço de repetição de matriz_valida.

3.6 Modo História Interativo

O Modo História Interativo, implementado no módulo “ModoHistoria”, usa como base o mito grego de Teseu e o Minotauro [1] para criar uma experiência interativa onde um jogador deve ajudar o herói a escapar do labirinto.

Teseu inicia em uma sala aleatória do labirinto e o jogador tem 5 tentativas, no total, para guiar o príncipe para fora do complexo, passando como input alguma das 4 próximas salas que Teseu deve ir. Note que, nesse modo, considera-se que o labirinto não possui nenhuma chave para ser coletada e que a posição marcada como “X” é a saída do labirinto.

A implementação segue esses passos:

- Inicialmente, é impressa na tela, com cores e pausas entre os textos, a história do modo e uma lista de instruções sobre como jogá-lo.
- Então, o algoritmo sorteia uma posição inicial para Teseu. Essa posição é considerada válida somente se existir pelo menos um caminho válido entre ela e a saída do labirinto, essa condição sendo verificada a partir da função “escolheCaminho”. Se forem sorteadas 500 posições e nenhuma delas for válida, o modo se encerra.
- Sabendo que existe pelo menos um caminho válido, usa-se a função “encontraTodosCaminhos” para guardar todos os caminhos válidos que um jogador pode seguir.

- Então, imprime-se o labirinto na tela, coleta-se o movimento do jogador e, a partir da lista com todos os caminhos possíveis, verifica-se se aquele movimento escolhido é válido. Se for, imprime-se o labirinto atualizado. Se não, imprime um aviso de caminho inválido e diminui a quantidade de tentativas restantes. Isso se repete para todos os movimentos do jogador.
- O jogo termina quando Teseu atinge a posição “X”, com o jogador recebendo uma mensagem de parabéns, ou quando o jogador erra o caminho pela quinta vez, recebendo uma mensagem de falha na missão.

3.7 Main

O Main do projeto implementa um menu de ações para o programa. Nele, o usuário consegue ler um labirinto de um arquivo de entrada, gerar um labirinto, encontrar um ou todos os caminhos válidos de um mapa, buscar um caminho válido que desconsidera as bordas do labirinto ou, ainda, testar o modo história. Imagens desse menu podem ser encontradas na seção 5 dessa documentação.

4. Compilação e Execução

O programa utiliza o utilitário Make para os processos de compilação e execução. Os comandos, dados a seguir, devem ser executados na pasta raiz do repositório, onde o arquivo “Makefile” está localizado:

- make: compila o programa no binário “main” e coloca-o na pasta “Main”.
- make run: executa o programa.

5. Resultados de Execução

Esta seção apresenta a execução do programa com quatro labirintos de entrada: os três arquivos de teste do trabalho (teste1.txt, teste2.txt e teste3.txt) e um quarto labirinto criado pelo gerador de labirintos e salvo em gerado.txt. Todos ficam na pasta Testes e seguem o mesmo formato de arquivo, de modo que podem ser carregados pela mesma opção do menu. A cada busca, o programa exibe o caminho encontrado, o número de chamadas recursivas, o tempo de execução e o nível máximo de recursão e, quando solicitado, anima o percurso em um painel de status com o passo atual e as chaves coletadas.

Os testes diferem no tamanho do mapa, no número de chaves e na origem do labirinto:

- **Teste 1:** labirinto de 10 por 10 posições (100 posições) com 2 chaves. É o menor dos mapas de teste, o que facilita visualizar e conferir todos os caminhos encontrados.
- **Teste 2:** labirinto de 22 por 22 posições (484 posições) com 3 chaves, cerca de 4,8 vezes maior que o Teste 1.
- **Teste 3:** labirinto de 35 por 35 posições (1.225 posições) com 4 chaves e muitas bifurcações e becos sem saída. É o mapa mais exigente para o backtracking.
- **Teste Gerado:** labirinto de 15 por 15 posições (225 posições) com 2 chaves e 35 peças de parede, criado pelo gerador de labirintos em vez de ser fornecido pronto.

Nos três mapas de teste, a entrada (A) fica no canto superior esquerdo e o baú (X) no canto inferior direito, de modo que o aventureiro precisa atravessar o labirinto de ponta a ponta. As paredes aparecem em azul, o caminho percorrido em verde e as chaves pela letra C.

O programa é controlado por menus. O menu inicial permite ler o labirinto de um arquivo (opção 1) ou gerar um novo (opção 2), como mostra a figura da seção 5.4. Depois que o labirinto é carregado, o usuário escolhe qual busca executar: um caminho válido, todos os caminhos válidos ou um caminho em labirinto sem bordas.

5.1 Menu

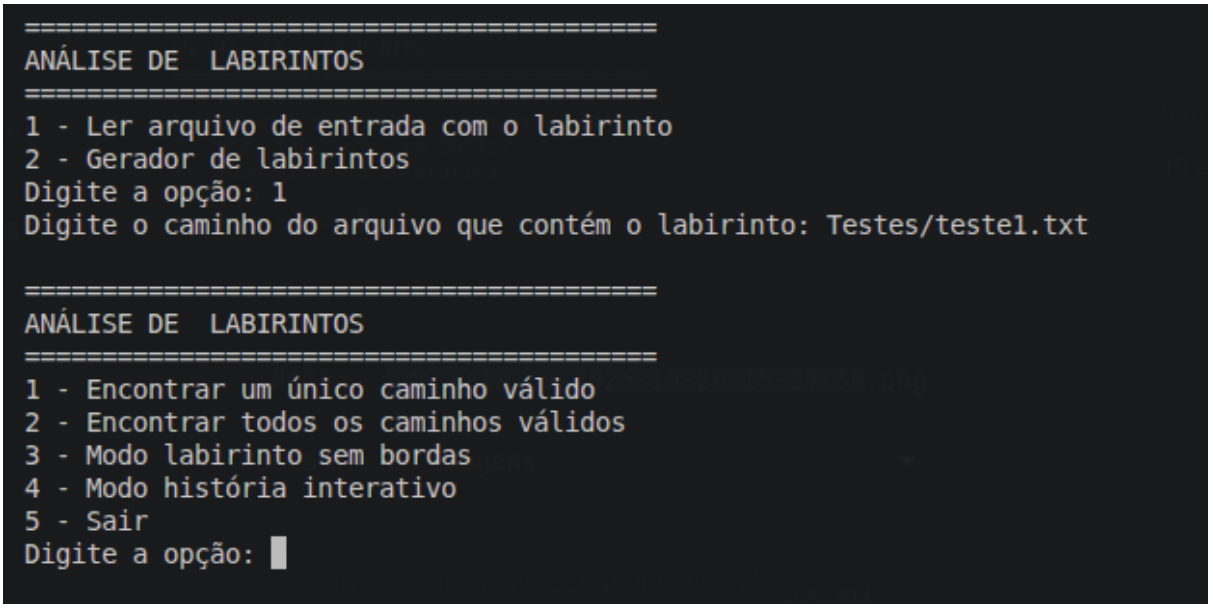


Figura 19: Menu inicial

Ao ser executado, o programa apresenta um menu em texto, intitulado ANÁLISE DE LABIRINTOS, que controla todo o fluxo de uso. O menu é dividido em duas etapas: na primeira, o usuário define de onde vem o labirinto e, na segunda, escolhe o que fazer com ele. A figura a seguir mostra as duas etapas em uma mesma execução.

Na primeira etapa, o menu oferece duas opções. A opção 1 lê o labirinto de um arquivo de entrada: o programa pede o caminho do arquivo, que é informado em relação à pasta raiz do projeto, como Testes/teste1.txt no exemplo da figura. A opção 2 aciona o gerador de labirintos, que cria um mapa novo a partir de parâmetros informados pelo usuário e o salva em Testes/gerado.txt, para ser carregado em seguida (seção 3.5).

Com o labirinto carregado, o programa exibe a segunda etapa do menu, com as operações disponíveis:

- 1 – Encontrar um único caminho válido: executa o backtracking da função escolheCaminho e para na primeira solução encontrada.
- 2 – Encontrar todos os caminhos válidos: executa a função encontraTodosCaminhos, que registra todas as soluções na lista de caminhos e, ao final, pergunta se o usuário quer ver algum deles de forma visual.

- 3 – Modo labirinto sem bordas: executa a busca em que o aventureiro pode sair por um lado do mapa e reaparecer no lado oposto.
- 4 – Modo história interativo: inicia o jogo baseado no mito de Teseu e o Minotauro, no qual o usuário guia o herói até a saída.
- 5 – Sair: encerra o programa.

5.2 Teste 1

O Teste 1 é o menor mapa e foi executado com as três buscas, o que permite compará-las em um caso que cabe inteiro na tela.

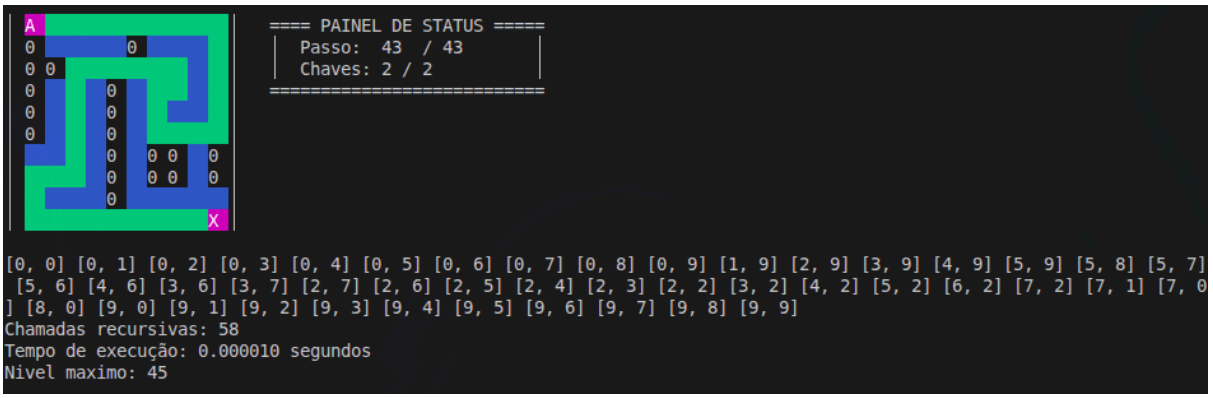


Figura 20: Teste 1 – busca de um caminho válido com escolheCaminho.

```
Digite a opção: 2
Caminho 1 (45 passos): [0, 0], [0, 1], [0, 2], [0, 3], [0, 4], [0, 5], [0, 6], [0, 7], [0, 8], [0, 9], [1, 9], [2, 9], [3,
 9], [4, 9], [5, 9], [5, 8], [5, 7], [5, 6], [4, 6], [3, 6], [3, 7], [2, 7], [2, 6], [2, 5], [2, 4], [2, 3], [2, 2], [3, 2
], [4, 2], [5, 2], [6, 2], [7, 2], [7, 1], [7, 0], [8, 0], [9, 0], [9, 1], [9, 2], [9, 3], [9, 4], [9, 5], [9, 6], [9, 7],
 [9, 8], [9, 9]

Caminho 2 (43 passos): [0, 0], [0, 1], [0, 2], [0, 3], [0, 4], [0, 5], [0, 6], [0, 7], [0, 8], [0, 9], [1, 9], [2, 9], [3,
 9], [4, 9], [5, 9], [5, 8], [5, 7], [5, 6], [4, 6], [3, 6], [2, 6], [2, 5], [2, 4], [2, 3], [2, 2], [3, 2], [4, 2], [5, 2
], [6, 2], [7, 2], [7, 1], [7, 0], [8, 0], [9, 0], [9, 1], [9, 2], [9, 3], [9, 4], [9, 5], [9, 6], [9, 7], [9, 8], [9, 9]

Caminho 3 (47 passos): [0, 0], [0, 1], [0, 2], [0, 3], [0, 4], [0, 5], [0, 6], [0, 7], [0, 8], [0, 9], [1, 9], [2, 9], [3,
 9], [4, 9], [5, 9], [5, 8], [5, 7], [6, 7], [6, 6], [5, 6], [4, 6], [3, 6], [3, 7], [2, 7], [2, 6], [2, 5], [2, 4], [2, 3
], [2, 2], [3, 2], [4, 2], [5, 2], [6, 2], [7, 2], [7, 1], [7, 0], [8, 0], [9, 0], [9, 1], [9, 2], [9, 3], [9, 4], [9, 5],
 [9, 6], [9, 7], [9, 8], [9, 9]

Caminho 4 (45 passos): [0, 0], [0, 1], [0, 2], [0, 3], [0, 4], [0, 5], [0, 6], [0, 7], [0, 8], [0, 9], [1, 9], [2, 9], [3,
 9], [4, 9], [5, 9], [5, 8], [5, 7], [6, 7], [6, 6], [5, 6], [4, 6], [3, 6], [2, 6], [2, 5], [2, 4], [2, 3], [2, 2], [3, 2
], [4, 2], [5, 2], [6, 2], [7, 2], [7, 1], [7, 0], [8, 0], [9, 0], [9, 1], [9, 2], [9, 3], [9, 4], [9, 5], [9, 6], [9, 7],
 [9, 8], [9, 9]

Caminho 5 (49 passos): [0, 0], [0, 1], [0, 2], [0, 3], [0, 4], [0, 5], [0, 6], [0, 7], [0, 8], [0, 9], [1, 9], [2, 9], [3,
 9], [4, 9], [5, 9], [5, 8], [5, 7], [6, 7], [7, 7], [7, 6], [6, 6], [5, 6], [4, 6], [3, 6], [3, 7], [2, 7], [2, 6], [2, 5
], [2, 4], [2, 3], [2, 2], [3, 2], [4, 2], [5, 2], [6, 2], [7, 2], [7, 1], [7, 0], [8, 0], [9, 0], [9, 1], [9, 2], [9, 3],
 [9, 4], [9, 5], [9, 6], [9, 7], [9, 8], [9, 9]

Caminho 6 (47 passos): [0, 0], [0, 1], [0, 2], [0, 3], [0, 4], [0, 5], [0, 6], [0, 7], [0, 8], [0, 9], [1, 9], [2, 9], [3,
 9], [4, 9], [5, 9], [5, 8], [5, 7], [6, 7], [7, 7], [7, 6], [6, 6], [5, 6], [4, 6], [3, 6], [2, 6], [2, 5], [2, 4], [2, 3
], [2, 2], [3, 2], [4, 2], [5, 2], [6, 2], [7, 2], [7, 1], [7, 0], [8, 0], [9, 0], [9, 1], [9, 2], [9, 3], [9, 4], [9, 5],
 [9, 6], [9, 7], [9, 8], [9, 9]

Chamadas recursivas: 1133
Tempo de execução: 0.000108 segundos
Nivel maximo: 54
Deseja ver, visualmente, algum dos caminhos? (S ou N):
```

Figura 21: Teste 1 – busca por todos os caminhos válidos com encontraTodosCaminhos.

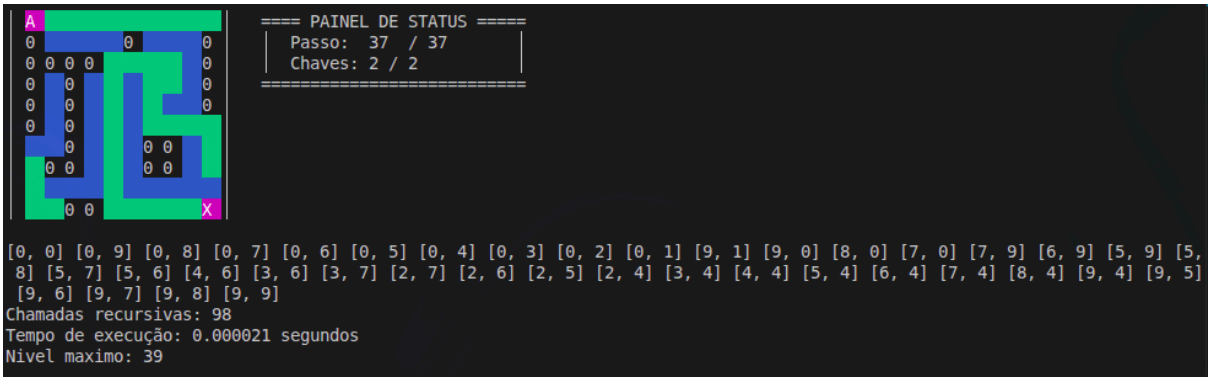


Figura 22: Teste 1 – Busca em labirinto sem bordas com Cobrinha.

5.3 Teste 2

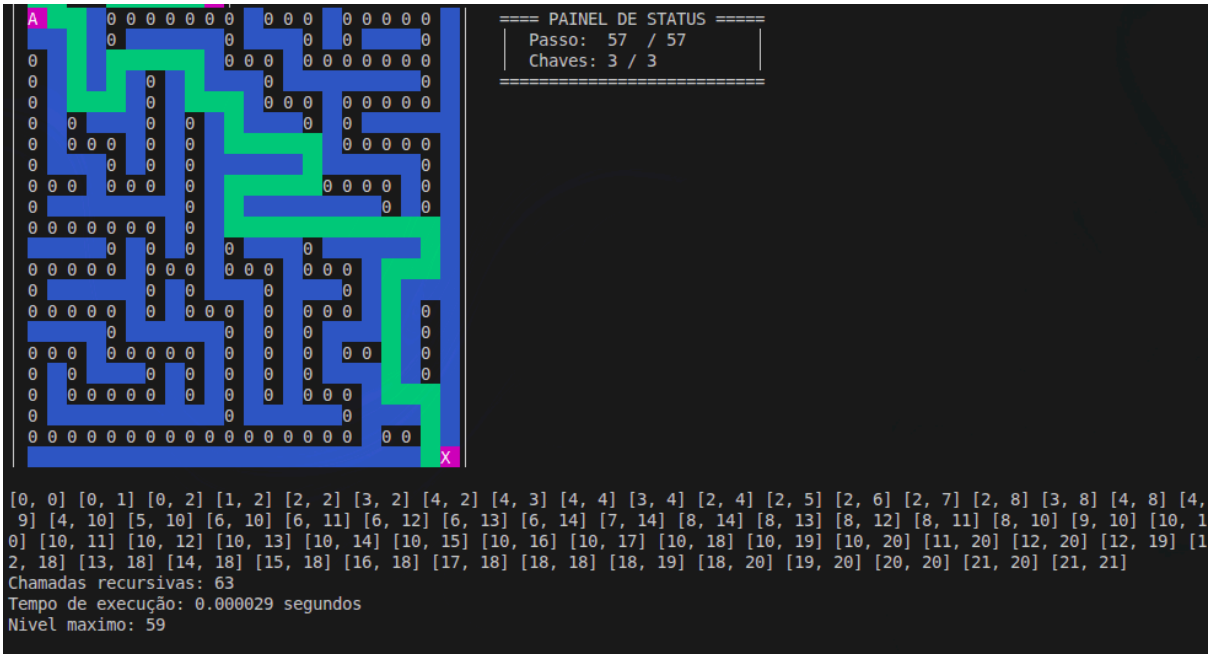


Figura 23: Teste 2 – Caminho encontrado e estatísticas da busca.

5.4 Teste 3

O Teste 3 é o maior mapa, com 35 por 35 posições e 4 chaves. A primeira figura a seguir mostra o último quadro da animação do caminho encontrado, com o painel de status indicando 295 passos de 295 e as 4 chaves coletadas, e a segunda mostra as estatísticas da busca.

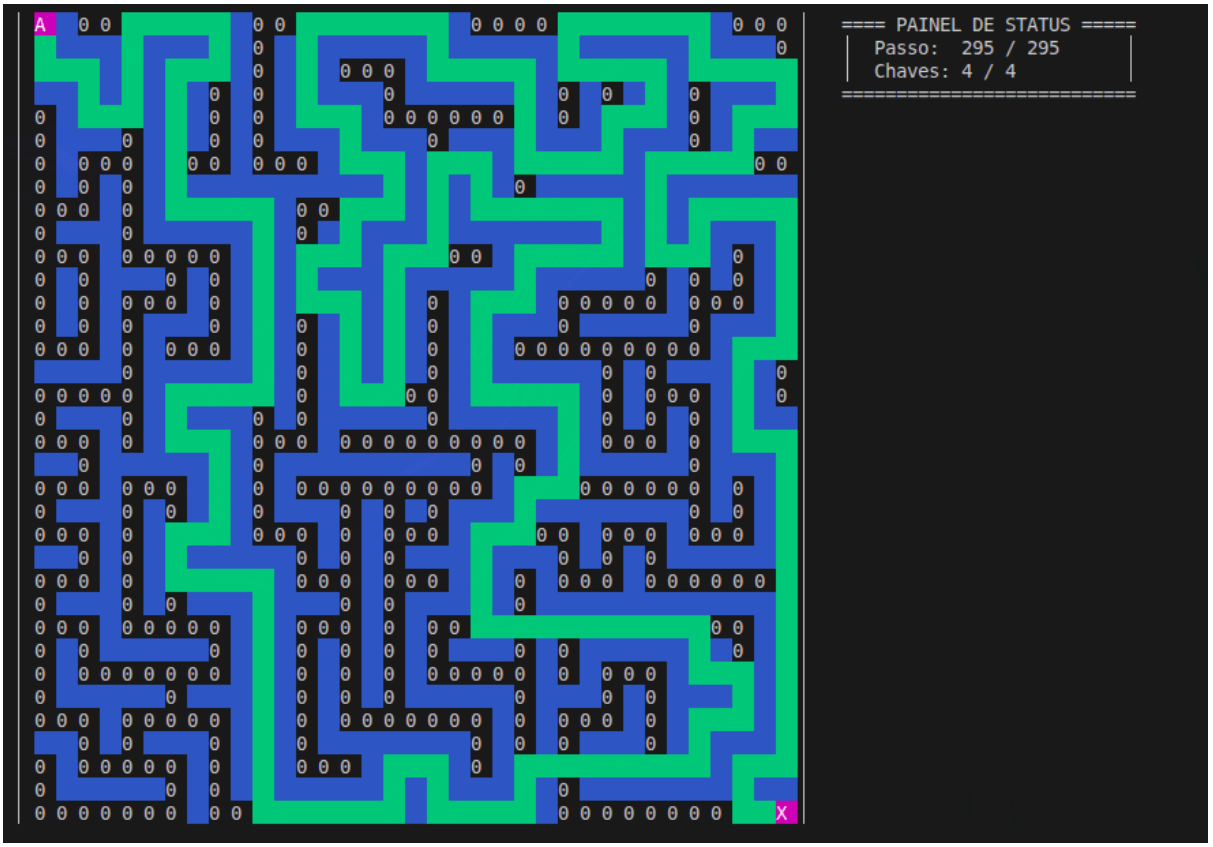


Figura 24: Teste 3 – último quadro da animação do caminho encontrado.

```
Chamadas recursivas: 108675498
Tempo de execução: 2.644830 segundos
Nivel maximo: 436
```

Figura 25: Teste 3 – estatísticas da busca com escolheCaminho.

5.5 Teste Gerado

O quarto teste usa um labirinto criado pelo gerador (seção 3.5) em vez de um arquivo pronto. A figura a seguir mostra o fluxo completo: no menu inicial, o usuário escolhe a opção 2 e informa 15 linhas e 15 colunas, 35 peças de parede e 2 chaves; o programa gera e exibe o labirinto, avisa que ele foi salvo em Testes/gerado.txt e, em seguida, pede o caminho do arquivo a ser carregado, ao qual o usuário responde com o próprio Testes/gerado.txt.

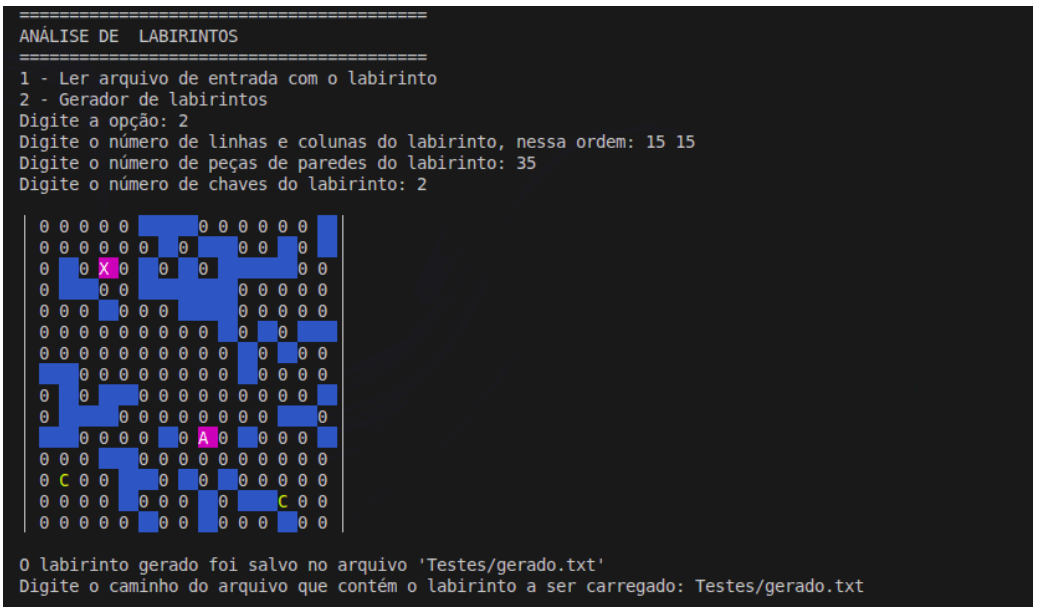


Figura 26: Menu inicial e geração do labirinto gerado.txt (15 por 15, 35 peças e 2 chaves).

Esse teste difere dos anteriores em três pontos. Primeiro, as paredes são formadas por blocos irregulares espalhados pelo mapa, e não por corredores estreitos, o que deixa o labirinto mais aberto e tende a produzir mais caminhos possíveis entre a entrada e o baú. Segundo, a entrada (A) e o baú (X) não estão nos cantos do mapa. Terceiro, ele valida a integração entre os módulos: o arquivo escrito pelo gerador é lido pelo mesmo código que lê os arquivos de teste, o que mostra que o formato produzido é compatível com o restante do programa.

6. Análise de Complexidade

6.2 Análise da função escolheCaminho

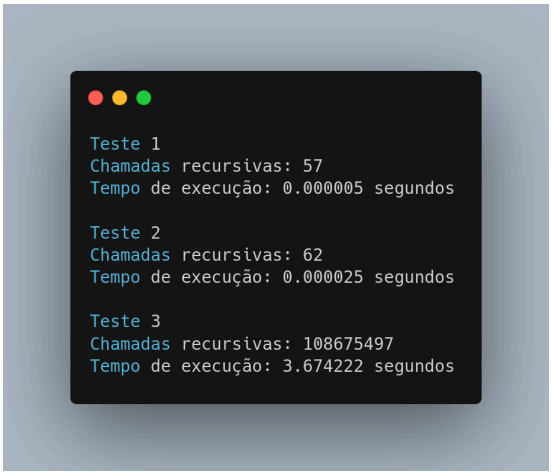


Figura 27: Resultados para a função escolheCaminho.

Comparando o comportamento da função em relação aos 3 testes é possível notar um aumento abrupto de tempo e número de chamadas recursivas do teste 3 (108.675.497 chamadas e 3,67 s), contra 57 chamadas no teste 1 e 62 no teste 2, enquanto o teste 3 (1225 posições) tem apenas cerca de 2,5 vezes mais posições que o teste 2 e faz milhões de vezes mais chamadas. A diferença de entrada de dados entre os testes 1 e 2 (considerando cada posição da matriz) é de quase 5 vezes maior para 2 e mesmo assim a quantidade de chamadas recursivas não tem uma diferença proporcional, isso acontece porque,mais importante que o tamanho da entrada é importante também levar em consideração a complexidade do mapa, bifurcações e caminhos errados aumentam muito mais o número de chamadas do que apenas o tamanho da entrada.

6. 2 Análise da função encontraTodosCaminhos

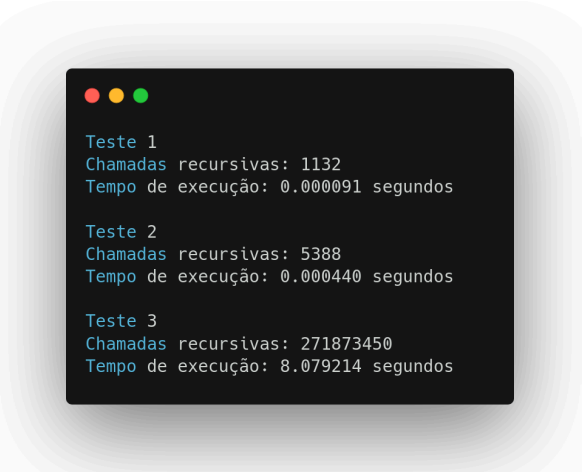


Figura 28: Resultados para a função encontraTodosCaminhos.

Aqui já existe uma mudança em relação a primeira função citada, podemos observar entre os testes 1 e 2 certo acompanhamento na proporção entre número de entradas e chamadas recursivas o crescimento da entrada acompanha o crescimento da quantidade de chamadas.

Embora nesse caso ocorra esse fenômeno, é possível ver a influência da complexidade do mapa pela diferença ainda fora do padrão do teste 3 para os outros.

O comportamento muda dessa forma pois por checar todos os possíveis caminhos ele se aproxima da exploração do mapa inteiro muito mais do que apenas encontrar o caminho que funciona.

6.3 Análise da função Cobrinha

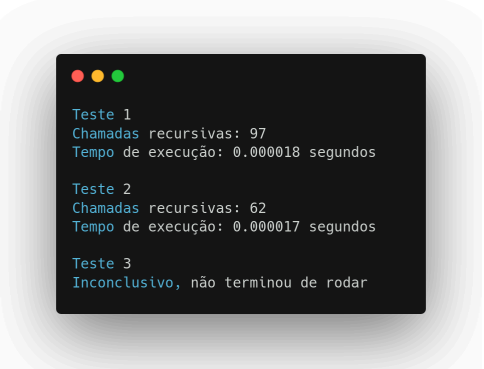


Figura 29: Resultados para a função Cobrinha.

Na função Cobrinha, que permite atravessar as bordas do mapa, os resultados diferem dos de escolheCaminho. No teste 1, o número de chamadas é de 97, contra 57 na versão sem bordas, pois as conexões extras entre as extremidades do mapa criam novas opções de movimento e aumentam as ramificações que o backtracking precisa explorar. No teste 2, o número de chamadas permanece em 62, igual ao obtido com bordas, o que indica que, nesse mapa, o caminho encontrado primeiro foi o mesmo e as conexões pelas bordas não foram necessárias para chegar ao baú, comprovado pela animação da impressão do caminho no escolheCaminho comparando com a animação na Cobrinha. Já o teste 3 não terminou a execução.

6.4 Nível máximo atingido



Figura 30: Resultados de nível máximo atingido.

O nível máximo corresponde à maior profundidade que a recursão alcançou, ou seja, o maior número de posições empilhadas no caminho em algum momento da busca, incluindo tentativas que acabaram em beco sem saída e foram desfeitas pelo backtracking.

Mais uma vez é possível notar a grande diferença entre o algoritmo que encontra todos para o que só encontra um dos caminhos, a profundidade dele é bem maior ao compararmos. Já o da cobrinha se se mostra no teste 1 menor e no 2 igual ao de encontraCaminhos, o que mostra que o comportamento dessa função

depende se o caminho sem bordas é mais claro e com menos bifurcações que o encontraCaminhos, resultado do teste 3 é inconclusivo.

7. Conclusão

Em suma, o trabalho envolveu conhecimentos técnicos e teóricos da área de Projeto e Análise de Algoritmos, e outros da área da programação, com destaque à Algoritmos e Estruturas de Dados. Dentre os quais estão a implementação de TADS, conhecimentos sobre backtracking, manipulação de memória dinâmica e de matrizes, e análise de complexidade de tempo e espaço. Todos esses assuntos foram ativamente aplicados para resolver o problema de encontrar um caminho válido dentro dos labirintos, respeitando suas características, bem como nas partes extras, que conseguiram ser resolvidas. Além disso, foi possível observar a criatividade por parte do grupo o qual implementou ideias originais respeitando as regras da tarefa, como a existência do modo história e a exibição do caminho percorrido no labirinto por meio de cores e de forma dinâmica.

8. Referências

[1] Mito de Teseu e o Minotauro